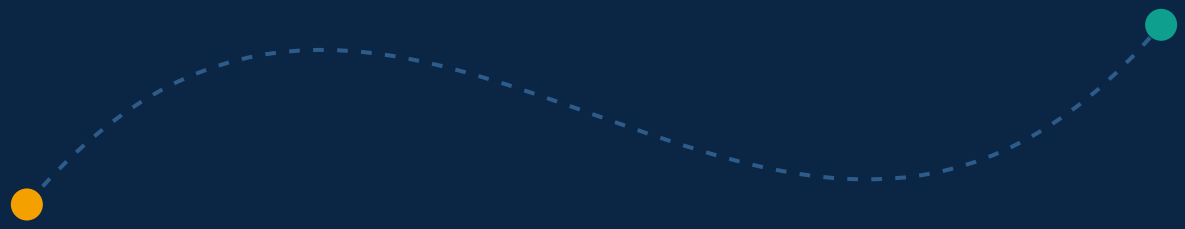

FlyPool

Share the cab. Split the fare. Skip the airport surge.

SYSTEM ARCHITECTURE & SCALING PLAN | v1

Prepared for: Riyanshi | Project: Flight_Home_Project

1. The idea

Airport par utarte hi cab ke price bahut high hote hain, par hamare paas koi option nahi hota. Yeh platform same airport par, same time window mein land hone wale aur **same direction** mein jaane wale logon ko match karta hai, taaki woh ek cab share karein aur fare split ho jaye.

Problem ke 4 asli challenges (API se bhi bade):

Challenge	Kyun important	Solution
Cold start	Match tabhi hoga jab same time par dusre users bhi hon	Launch **Delhi IGI (T3)** se, phir Bengaluru BLR. Pehle 1-2 popular corridors. Flight se 1 din pehle bhi request allowed.
Safety	Strangers ke saath cab; trust nahi to koi use nahi karega	Phone OTP, ID verify, women-only option, report/block, trip share, exact address match ke baad.
Matching quality	Galat match = kharab experience	Time window + destination distance + group size + luggage.
Cab booking	Hum cab fleet nahi chalate	MVP: group + fare split. Cab Uber/Ola ya prepaid counter se. Baad mein deep link.

2. High-level system architecture

Ek **modular monolith** se shuru karte hain: ek deploy, clean module boundaries. Load badhne par matching aur chat ko alag service bana denge.

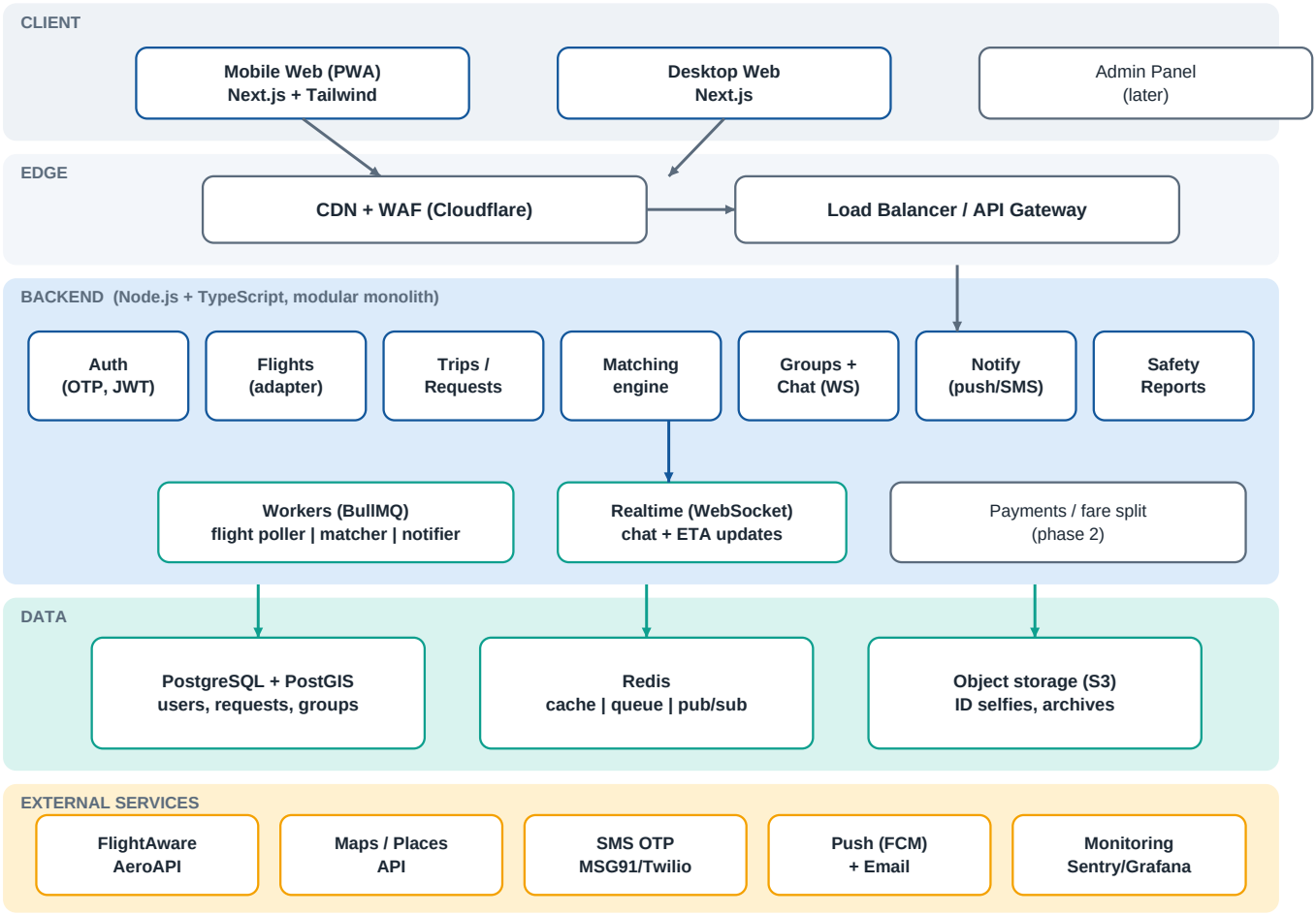

Figure 1: Complete system architecture (client, edge, backend, data, external).

3. Flight API: kaun si use karein

API	Good for	Cost / free tier	Verdict
Aviationstack	Simple flight status, airline, airport data	Free: 100 req/month. Paid from $49.99/month	**Prototype only.** Free tier ek din mein khatam.
FlightAware AeroAPI	Live tracking, delays, predictive ETA	Free Starter 5 queries/min. Paid from $25/month; higher tiers need minimums	**BEST CHOICE.** Production-grade ETA/delay.
OpenSky Network	Arrivals by time window; research use	Free REST API	**Avoid.** Non-commercial only; hamara product commercial hai.
Amadeus / Cirium / OAG	Airline-grade schedules, disruptions	Enterprise / custom pricing	**Later,** jab scale + contract chahiye.

Recommendation: Dev = Aviationstack/AeroAPI free. Launch = AeroAPI paid. Scale = Cirium/OAG. Pricing change hoti rehti hai, isliye current rates unki website par verify karein.

Paid plan kaise lein (step by step)

- FlightAware AeroAPI portal par account banao aur plan choose karo.
- Card add karo; API key milegi.
- Key sirf **backend ke .env** mein rakho. Frontend ya GitHub par kabhi nahi.
- Usage limit aur billing alert set karo taaki surprise bill na aaye.
- Terms check karo: commercial use allowed ho. Free personal tier ko commercial product mein mat use karo.

Cost bachane ki trick

Hum *airport par sab flights* nahi mangte. User flight number + date deta hai, hum **sirf us ek flight** ko lookup karte hain aur Redis mein cache karte hain (key = `flight:{no}:{date}`). 50 users ek hi flight par = sirf **1 API call**. Landing ke paas poll interval 10-15 min rakho, baaki time nahi.

Hamesha ek `FlightProvider` adapter interface rakho, taaki provider badalna ek file ka kaam ho.

4. User journey aur request lifecycle

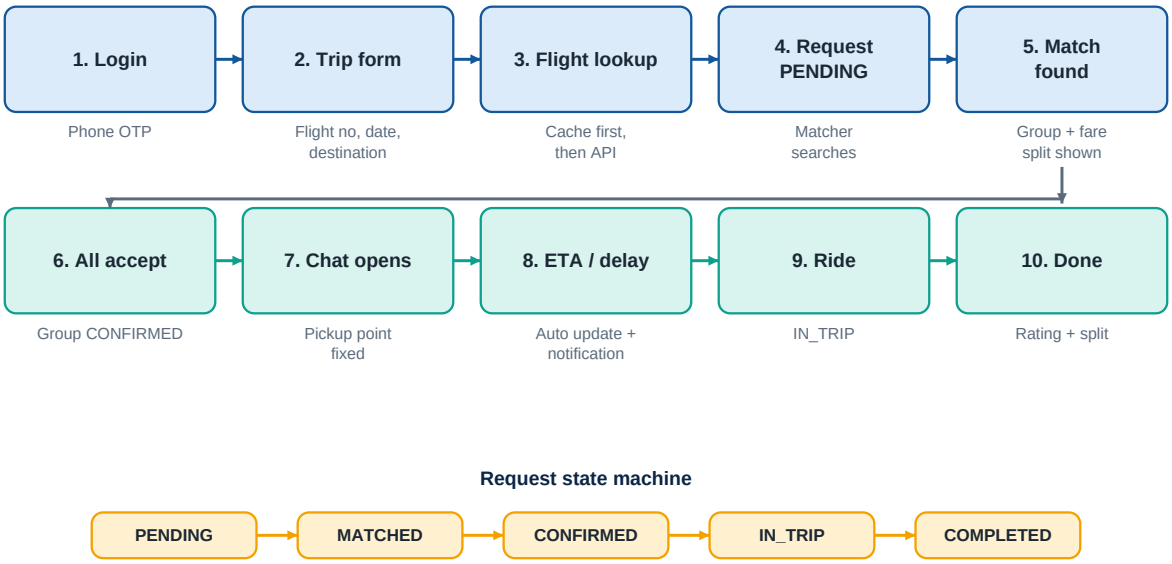

Figure 2: End-to-end user journey and request state machine.

5. Matching algorithm

Do requests tab compatible hain jab:

- **Same airport** (ideally same terminal).
- **Time:** landing ETA ka farak lagbhag 40 minute ke andar. Flight delay par re-evaluate.
- **Destination:** 3-5 km radius (PostGIS ST_DWithin ya H3 cell). V2 mein route-detour check.
- **Group size** cab capacity ke andar (sedan 3 seats, SUV 5).
- **Preferences** (women-only etc.) hard filter.

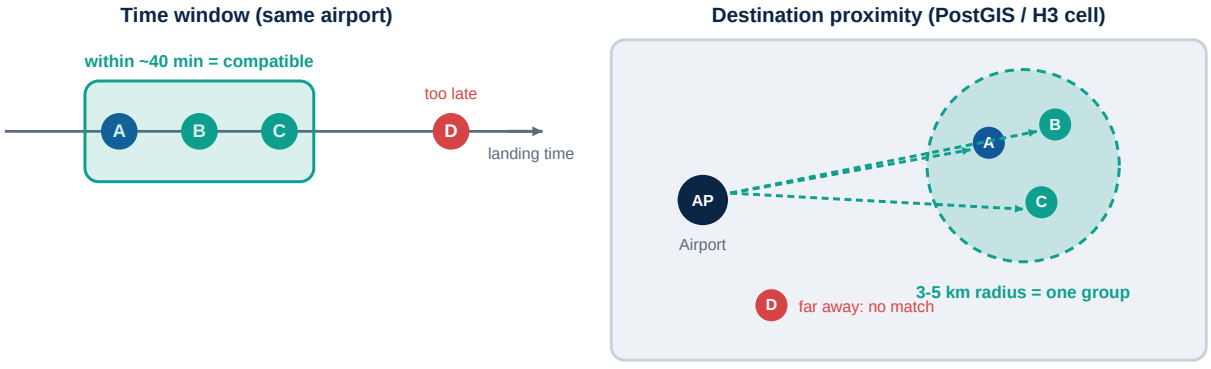

Figure 3: Time-window and destination-proximity matching.

Score = weighted(time gap, destination distance, detour minutes). Matcher ek **background worker** mein chalta hai (naye request par aur har 2 min), request-time par heavy kaam nahi. Index: `(airport_id, eta, status)`.

6. Database design

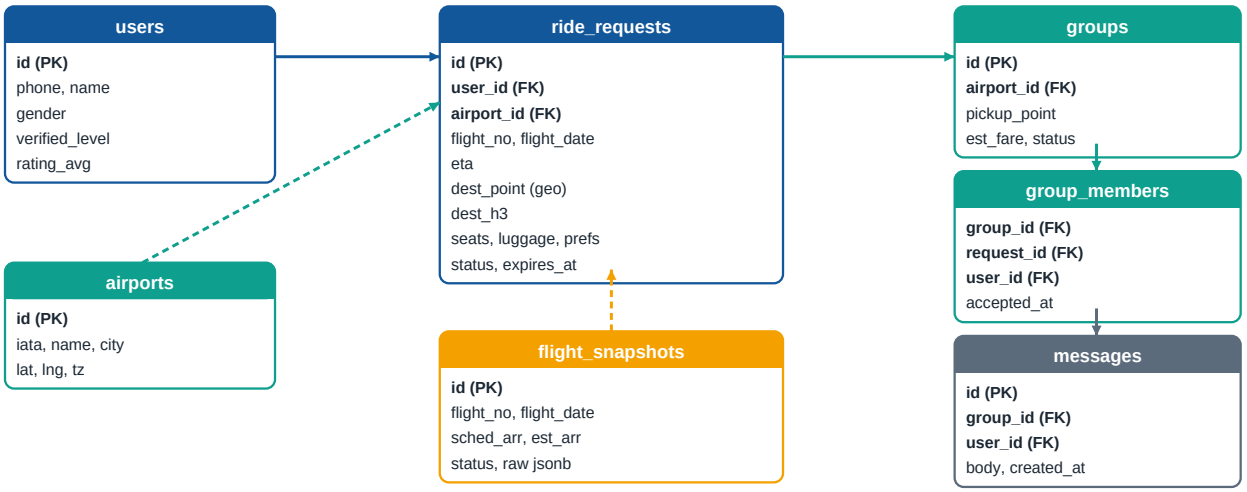

Figure 4: Core data model (ER overview).

Rules jo day-1 se follow karne hain

- UUID/ULID primary keys (sharding-friendly). Auto-increment mat lo.
- Relevant tables mein **airport_id** rakho: future shard key.
- Unique `(user_id, flight_no, flight_date)` taaki duplicate request na ho.
- Ephemeral data par `expires_at` + cleanup job.
- GIST index on dest_point; index on (airport_id, eta, status).

7. Scalability plan

Is app ka data **time-boxed** hai: aaj ki requests kal bekaar. Isse scaling aasan hoti hai, aur natural shard key **airport** hai kyunki ek airport ka data dusre se kabhi mix nahi hota.

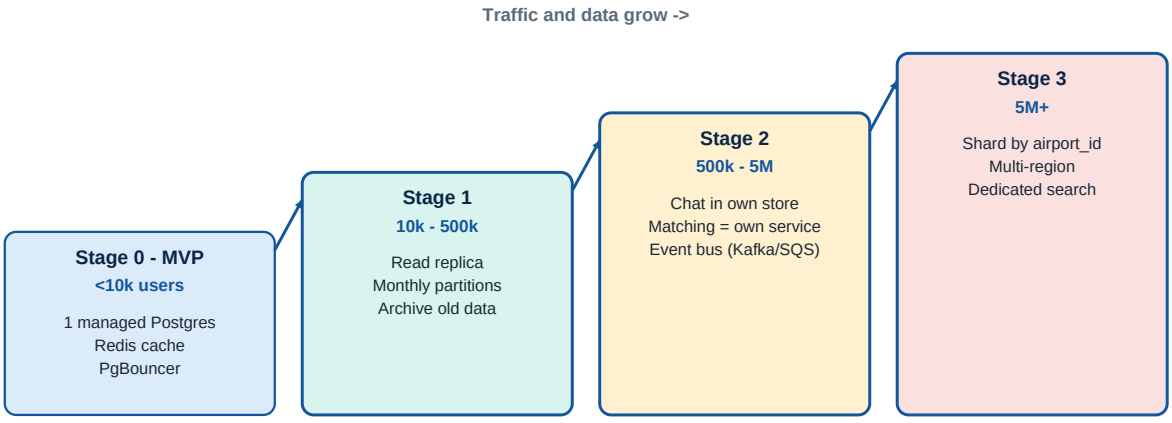

Figure 5: Database and service scaling stages.

App tier scaling

- Stateless backend (JWT) + containers with autoscaling (ECS / Cloud Run).
- WebSocket scale: Redis pub/sub adapter, taaki multiple instances ke beech chat chale.
- Workers (flight poller, matcher, notifier) alag scale karo.

- CDN se static + images; Redis rate limiting har endpoint par (OTP par strict).

Tech stack

Layer	Choice	Why
Frontend	Next.js + TypeScript + Tailwind, PWA	Mobile par app jaisa, har screen par responsive
Backend	NestJS (Node.js, TypeScript)	Same language, modular structure
Database	PostgreSQL + PostGIS (Supabase/Neon/RDS)	Relational + location queries
Cache/Queue	Redis + BullMQ	Flight cache, jobs, pub/sub
Auth	Phone OTP + JWT (Google optional)	India mein phone primary
Maps	Google Places / Mapbox / Ola Maps	Autocomplete + distance
Hosting	Vercel + Railway/Render, later AWS/GCP	Fast start, scale later

8. Safety, privacy aur legal (India)

Area	Measure
Identity	Phone OTP mandatory; baad mein DigiLocker / selfie verify badge
Rider comfort	Women-only / women-prefer matching option
Privacy	Exact address sirf confirmed group ko; numbers masked; in-app chat
Incidents	Report/block, emergency button, live trip share link
Law	DPDP Act 2023: consent, data minimisation, delete-account. Terms mein liability clear karo (lawyer se verify).

9. Build roadmap

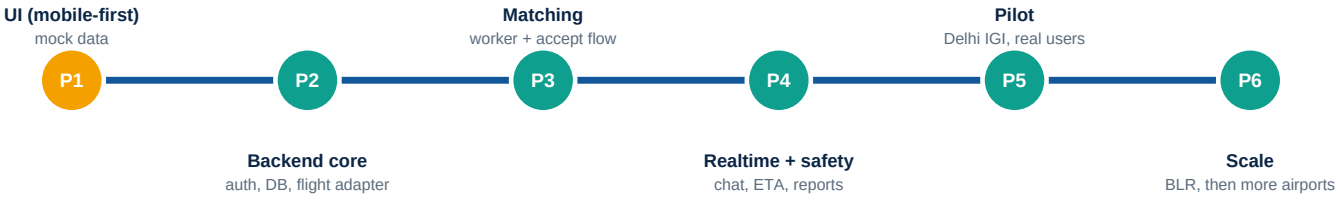

Figure 6: Phase-wise build plan.

Monetisation (baad mein)

- Per-successful-match chhoti fee ya premium priority matching.
- Cab partners / affiliate deep links (Uber, Ola, airport taxi).
- Airline, airport aur hotel partnerships.

Key metrics: match rate, time-to-match, average savings per rider, repeat usage, safety incidents.